

Network Science Data II

CNET 5051, Fall 2026

Dr. Milo Z. Trujillo

Assignment 1, due Monday, Oct. 12th, 11:00pm EST.

Please submit a .pdf of your answers via Canvas, including a url to your .ipynb notebook in a public GitHub repository. Please name your Canvas submission as Lastname.pdf, (*e.g.* Klein.pdf). If you need help with LaTeX or GitHub, please visit during office hours or post questions to the Slack channel.

0. *Question 0 (0 points).*

Include your code in a (well-documented, commented, clear) `assignment01.ipynb` file. Scores for the code-based questions below will be based on your written responses *and* code.

1. *Question 1 (20 points).*

There are a lot of ways to store network data. Each format generally has advantages and disadvantages that make them suited to particular types of situations. Let's explore the differences between storing network data in a `.csv`, `pickle`, or `.gml` file, and in Elasticsearch, which is commonly used for graph database applications. For your research, look at the documentation for each approach, seek out StackOverflow threads, and consider consulting the Bagrow & Ahn textbook.

- (a) What are the advantages of each storage approach?
- (b) What are the disadvantages of each storage approach?
- (c) At least one of these storage approaches poses a non-trivial security risk. Which one(s) should we worry about, and what is the security risk? Please explain this as simply as possible.
- (d) For each storage approach, think of a scenario where that storage approach would be an optimal choice. Explain your reasoning.

2. *Question 2 (20 points).*

Algorithm 11.1 in Bagrow & Ahn, the basic edge exchange algorithm, modifies a graph in place to produce a null model while preserving the degree distribution. It does so by picking two edges at random and swapping their endpoints.

```
1: procedure EDGE_EXCHANGE( $G, t$ )
2:    $s \leftarrow 0$ 
3:   while  $s < t$  do
4:      $(i, j) \leftarrow \text{get\_random\_edge}(G)$ 
```

```

5:      (u, v) ← get_random_edge(G)
6:      if i = u or i = v or j = u or j = v then
7:          continue
8:      end if
9:      if has_edge(G, i, v) or has_edge(G, j, u) then
10:         continue
11:      end if
12:      remove_edge(G, i, j)
13:      remove_edge(G, u, v)
14:      add_edge(G, i, v)
15:      add_edge(G, j, u)
16:      s ← s + 1
17:   end while
18:   return G
19: end procedure

```

- (a) (Bagrow & Ahn Question 11.2) The basic edge exchange algorithm (as shown above) will enter an infinite loop and never end if no allowable exchanges are present. Describe a scenario where you will never be able to find an allowable edge exchange according to this algorithm.
- (b) (Bagrow & Ahn Question 11.3) Modify the algorithm to prevent an infinite loop by abandoning the exchange process after reaching a given number of attempts without a successful exchange.
- (c) (Bagrow & Ahn Question 11.4) Modify the algorithm to preserve not only the degrees of every node but also the overall connectivity of the network. How does this additional constraint make edge exchange more difficult?
- (d) (Bagrow & Ahn Question 11.5) Suppose your network has a binary attribute $x_i \in \{-1, 1\}$ associated with every node i . Modify the algorithm to preserve the correlation on edges of x . This means that any new edge (a, b) inserted in place of an existing edge (i, j) must have the same attributes $(x_i, x_j) = (x_a, x_b)$. How might this additional constraint make edge exchange more difficult?

3. Question 3 (20 points).

There are many ways to measure a node's centrality in a network. Pick one centrality measure out of {betweenness centrality, harmonic centrality, percolation centrality} and do the following:

- (a) Write your own function to compute a single node's centrality in a graph. This function should take a `networkx` graph and one node in the graph as input. As output, it should return a float or integer representing the node's centrality in the graph according to one of the centrality measures listed above.
- (b) Augment your function with error handling. It should be able to return useful messages when given any of the following:
 - i. A node that is not in the graph

- ii. A graph that is not a `networkx` object
 - iii. A graph with no connectivity (or not enough connectivity to produce a valid centrality result)
 - iv. Any situation that might produce numerical difficulties (e.g. dividing by zero) for your chosen centrality measure
- (c) Compare your function to the corresponding `networkx` function across several graphs you design as test inputs, including one with no connectivity. Does your function ever return different results from the `networkx` function? Why do you think this is the case?
- (d) Putting it all together: Write a function that takes a centrality function as input and **tests** the function for robustness to edge cases (like a graph with no connectivity) and correctness of outputs. Explain why you selected the test cases you chose.

4. *Question 4 (20 points).*

Download a network dataset from <https://networkrepository.com/> with between $1,000 < N < 20,000$ nodes, and explore it as described below.

- (a) In 2-5 sentences, describe your network. What are the nodes, what are the links, what domain is it from? Be sure to include appropriate citation(s) to the original dataset or research paper, if relevant.
- (b) Compare your network's properties to those of a *randomized* network, by selecting a network null model to compare against. Your null model should preserve the degree distribution of the network, so randomize your network using *e.g.* a configuration model or degree-preserving network rewiring (see Class 4).
 - i. Create a figure with three subplots, corresponding to three distributions of your network; write brief interpretations of what you observe. These can be distributions of node properties (i.e. degree, centrality measures, etc.) or edge properties (i.e. edge weights). Be sure to bin these distributions appropriately.
 - ii. Create an analogous figure for the randomized version of your network; write brief interpretations of what you observe.
- (c) Discuss the similarities and/or differences between your network dataset and the randomized versions of this network. How does this exercise—comparing network data to a null model—help us gain insights from network analysis?

5. *(Optional) Question 5 (5 bonus points).*

For every problem set this semester, there will be an opportunity to earn extra credit. Since this is a fairly new course (of a new masters program), we are eager to receive genuine feedback on the material, pace, instruction style, etc. of the course so far. Note: Learning how to give effective feedback is an important skill during your PhD, so we hope this is an opportunity for you to skill-build *and* improve the course for future students. (Also, it's a bit odd to be evaluated on on how good your feedback

is, so we will be generous with awarding points for these. With that said, please take some time to provide us with productive comments!) If you need resources for giving effective feedback, here are a few:

- <https://hbr.org/2017/10/how-to-give-feedback-people-can-actually-use>
- <https://www.sessionlab.com/blog/feedback-techniques/>